中科院计算所
INSTITUTE OF COMPUTING TECHNOLOGY,CAS

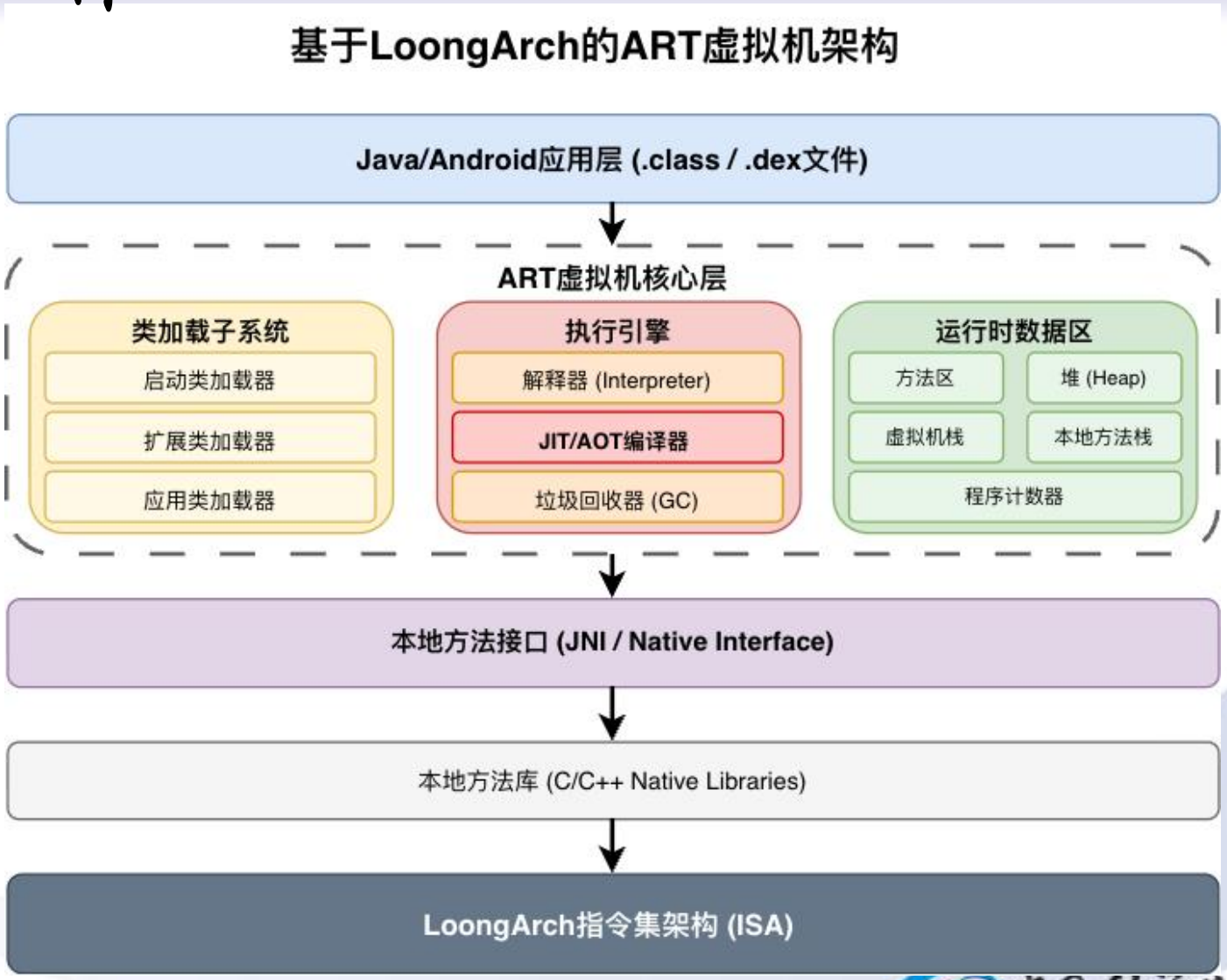
基于LoongArch架构的ART虚拟机执行引擎适配与优化技术研究

鄢玺

导师：张福新

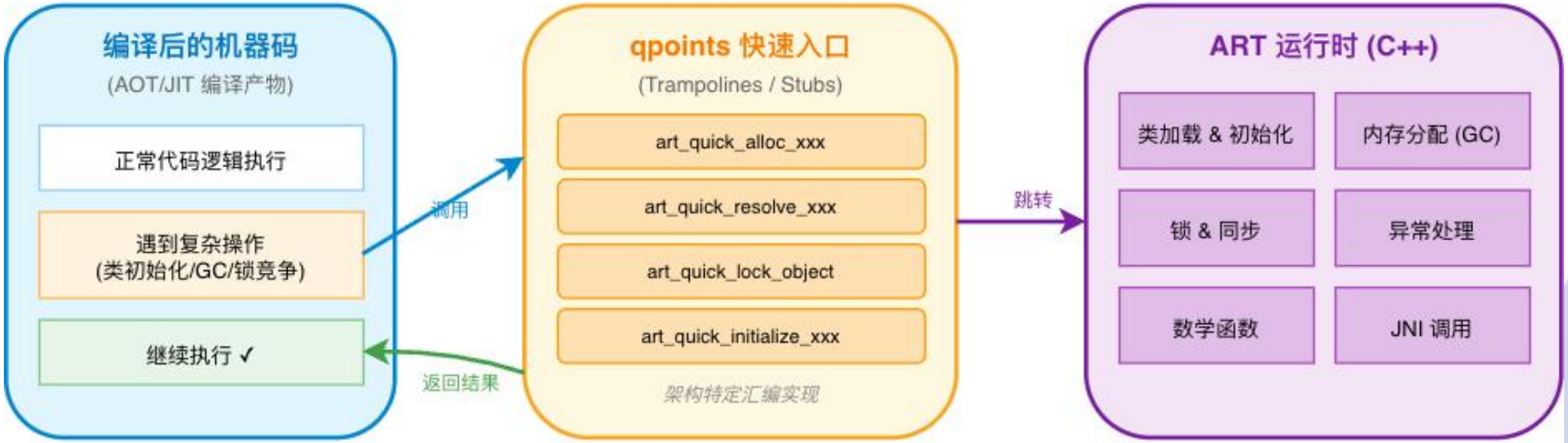
2025年12月29日

适配工作


基于LoongArch的ART虚拟机架构
Java/Android应用层 (.class / .dex文件)
ART虚拟机核心层
类加载子系统
启动类加载器
扩展类加载器
应用类加载器
执行引擎
解释器 (Interpreter)
JIT/AOT编译器
垃圾回收器 (GC)
运行时数据区
方法区
堆 (Heap)
虚拟机栈
本地方法栈
程序计数器
本地方法接口 (JNI / Native Interface)
本地方法库 (C/C++ Native Libraries)
LoongArch指令集架构 (ISA)

特殊机制

ART qpoints：编译代码与运行时的快速路径机制



对比：传统方式 vs qpoints

✗ 传统 Deoptimization

编译代码 → 销毁栈帧 → 切换到解释器 → 处理 → 重建状态

开销大：上下文切换 + 状态重建

✓ qpoints 快速路径

编译代码 → 直接调用 qpoint 函数 → 处理 → 返回继续执行

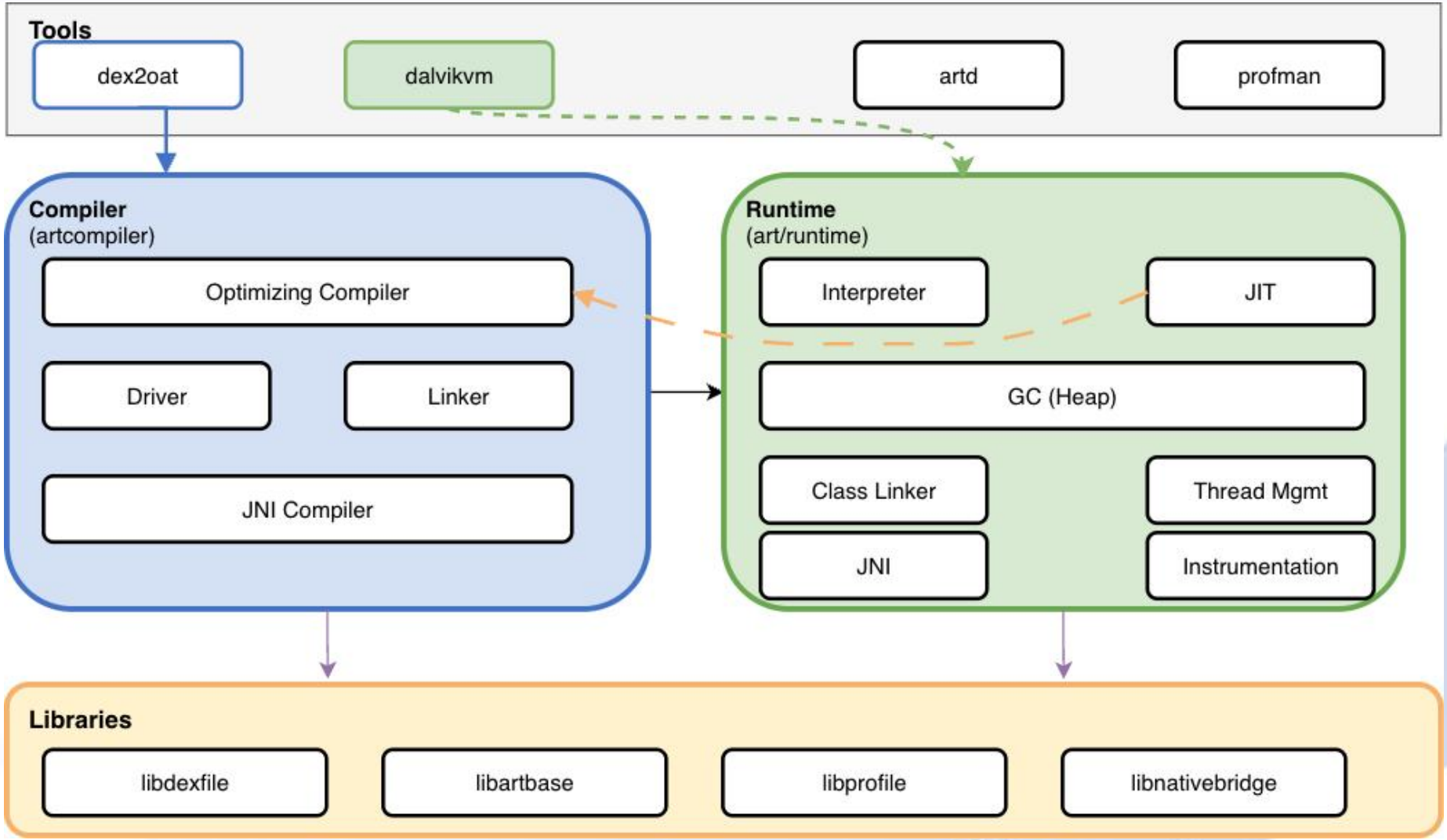
高效：仅一次函数调用，无上下文切换

核心价值：qpoints 让编译后的代码能以最低开销（一条跳转指令）调用底层 C++ 运行时能力，同时允许手写汇编代码进行特定架构优化来替换底层C++实现

提纲

二、系统优化当前进展

主体架构


Tools
dex2oat
dalvikvm
artd
profman
Compiler
(artcompiler)
Optimizing Compiler
Driver
Linker
JNI Compiler
Runtime
(art/runtime)
Interpreter
JIT
GC (Heap)
Class Linker
Thread Mgmt
JNI
Instrumentation
Libraries
libdexfile
libartbase
libprofile
libnativebridge

DexFileVerifier介绍

- Android ART (Android Runtime) 中的一个核心组件
  负责在加载Dex文件之后，验证其合法性和完整性

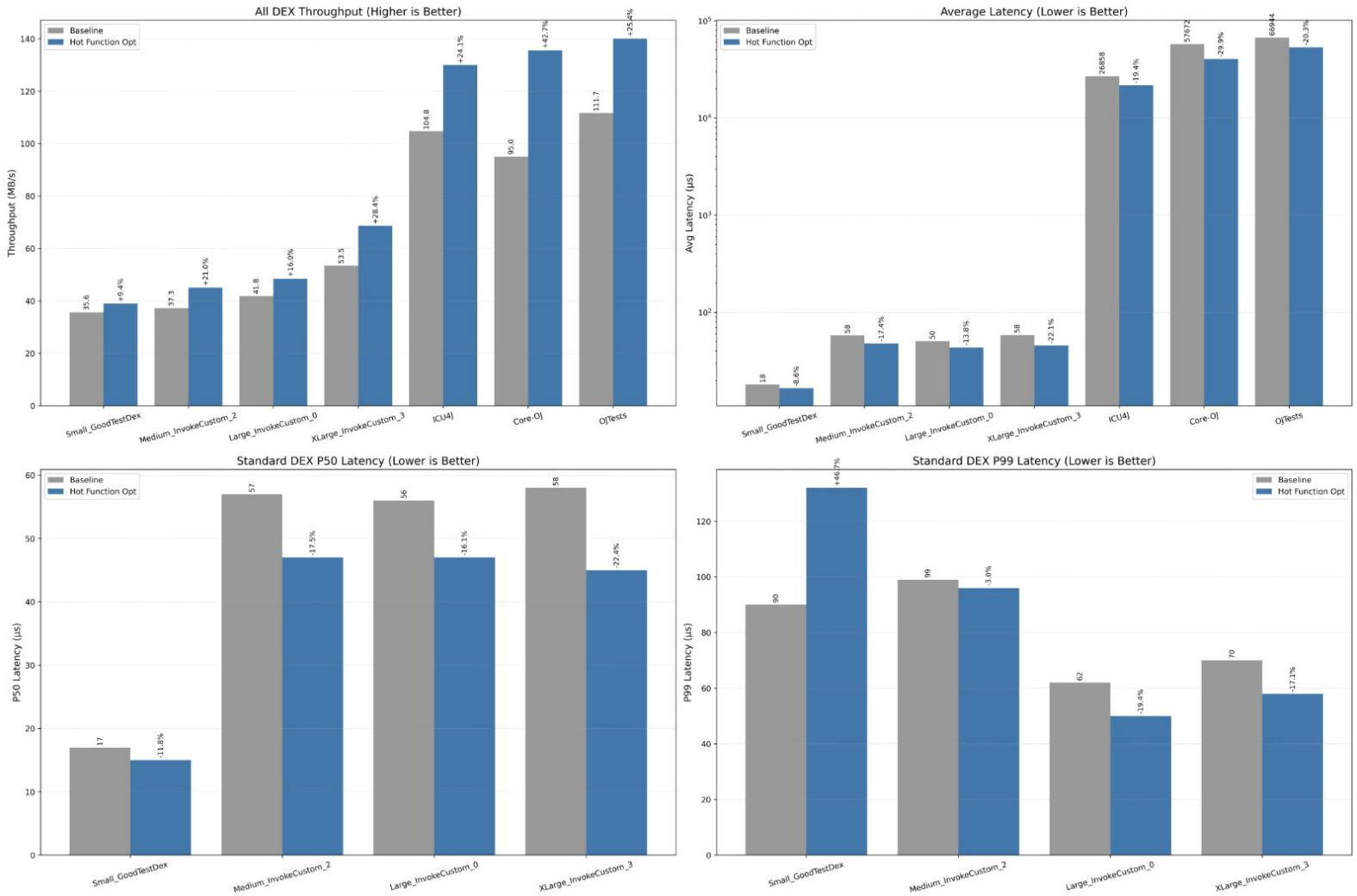
- 测试文件DEX
- 测试指标：吞吐率、延迟、P50/99延迟

File Name	Size	Type
Small_GoodTestDex	676B	Standard DEX
Medium_InvokeCustom_2	2268B	Standard DEX
Large_InvokeCustom_0	2212B	Standard DEX
XLarge_InvokeCustom_3	3276B	Standard DEX
ICU4J	2.8 MB	Real-world DEX
Core-OJ	5.5 MB	Real-world DEX
OJTests	7.5 MB	Real-world DEX

热点函数优化

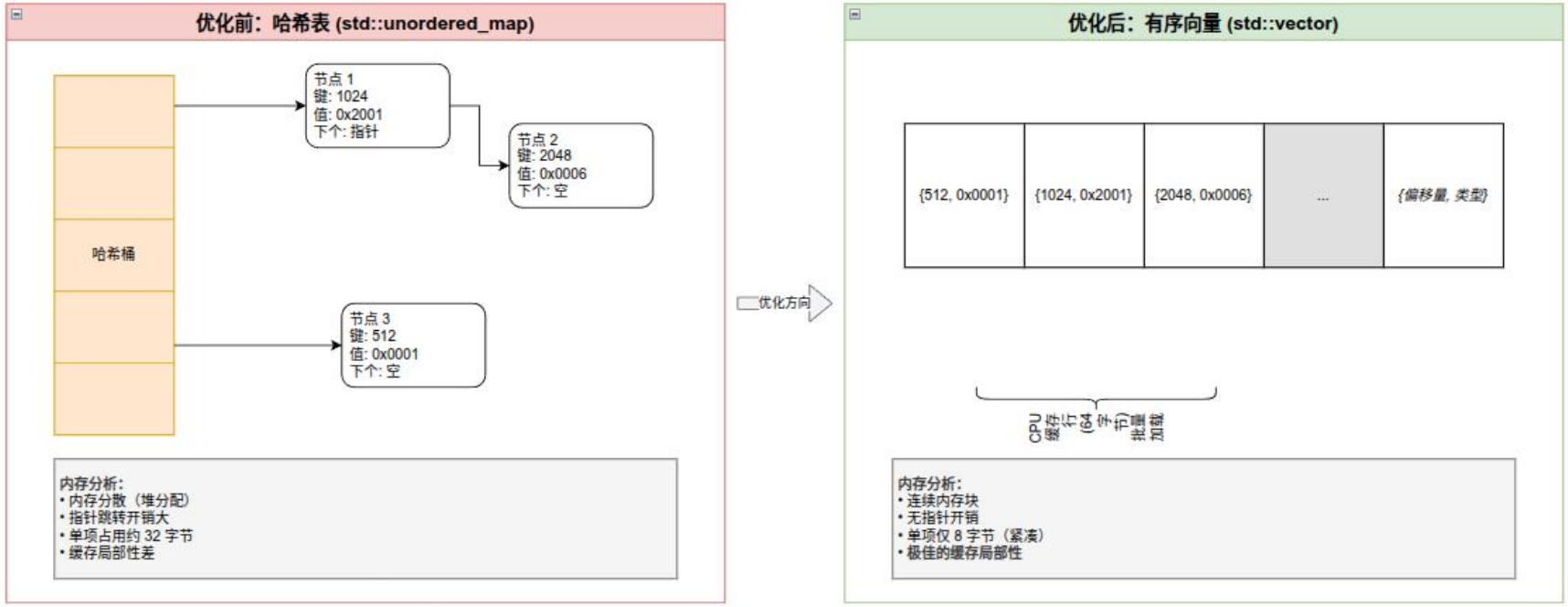
- 文件边界缓存
  - 每次调用 `EndOfFile()` 都需要通过 `Begin() + size_` 计算，产生函数调用开销
  - 在构造函数中一次性计算 file_end_，后续直接使用缓存值
  - 消除字符串验证等热路径中的重复计算
- HashMap预分配
  - HashMap 默认容量小，在插入大量数据时会触发多次 rehash，导致内存重分配和数据迁移
  - 根据 DEX header 中的 class/method/field 数量预估容量并预分配
  - 消除 rehash 开销，保证 O(1) 插入性能
- 热函数内联
  - 小函数调用开销（栈帧创建、参数传递、返回）在热路径上累积可观
  - 编译器内联优化，直接展开函数体
  - 消除函数调用开销，改善 CPU 分支预测

热点函数优化效果

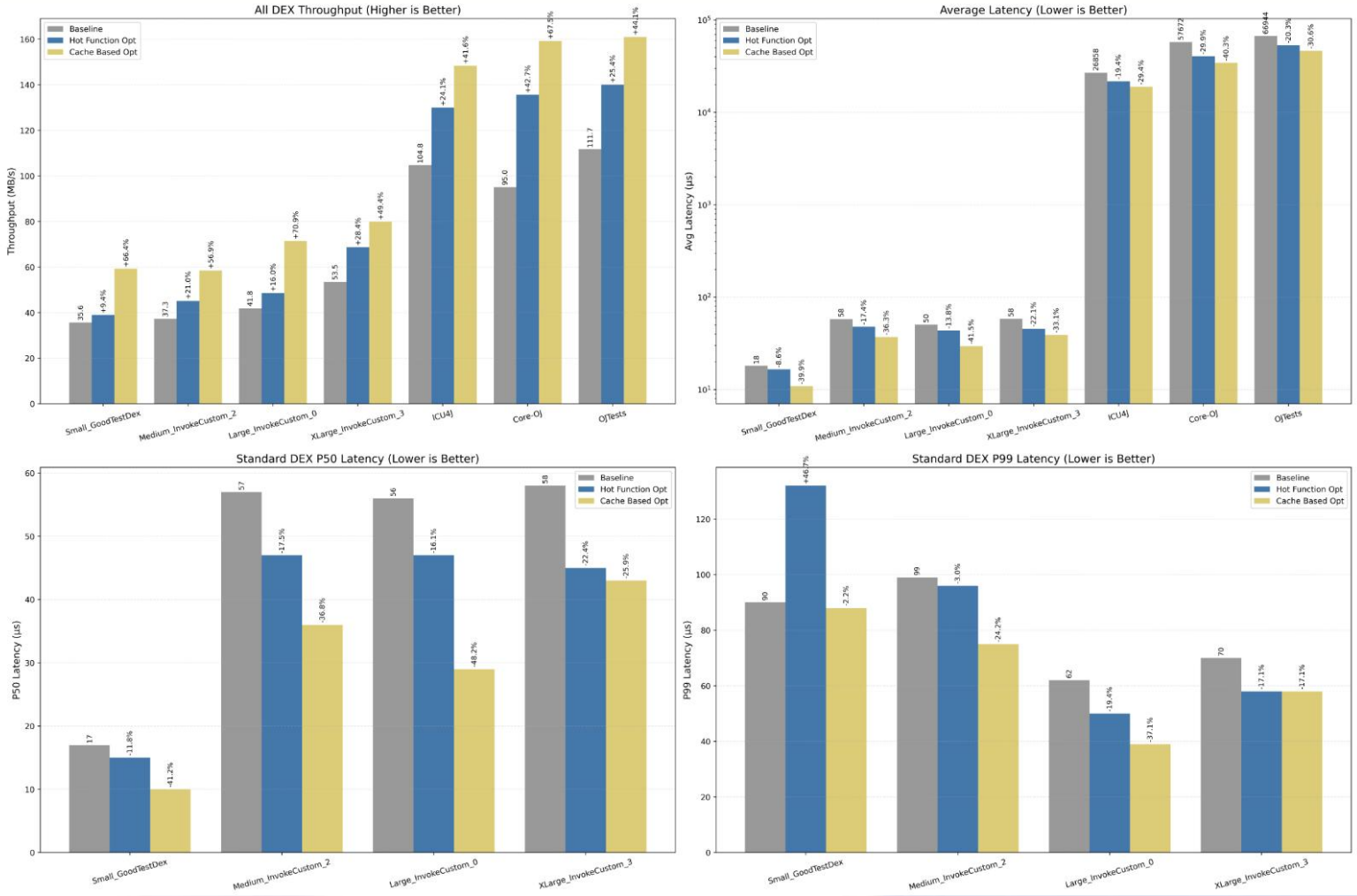


- 吞吐量: ↑ 23.9%
- 平均延迟: ↓ 18.8%
- p50: ↓ 16.9%
- p99: ↓ 1.8%

基于缓存局部性的数据结构优化



基于缓存局部性的数据结构优化效果（累计）



本次优化后

- 吞吐量: ↑ 56.7%
- 延迟: ↓ 35.9%
- p50: ↓ 38%
- p99-:↓ 20%

上次优化后

- 吞吐量: ↑ 23.9%
- 平均延迟: ↓ 18.8%
- p50: ↓ 16.9%
- p99: ↓ 1.8%

基于查找表的字符串处理

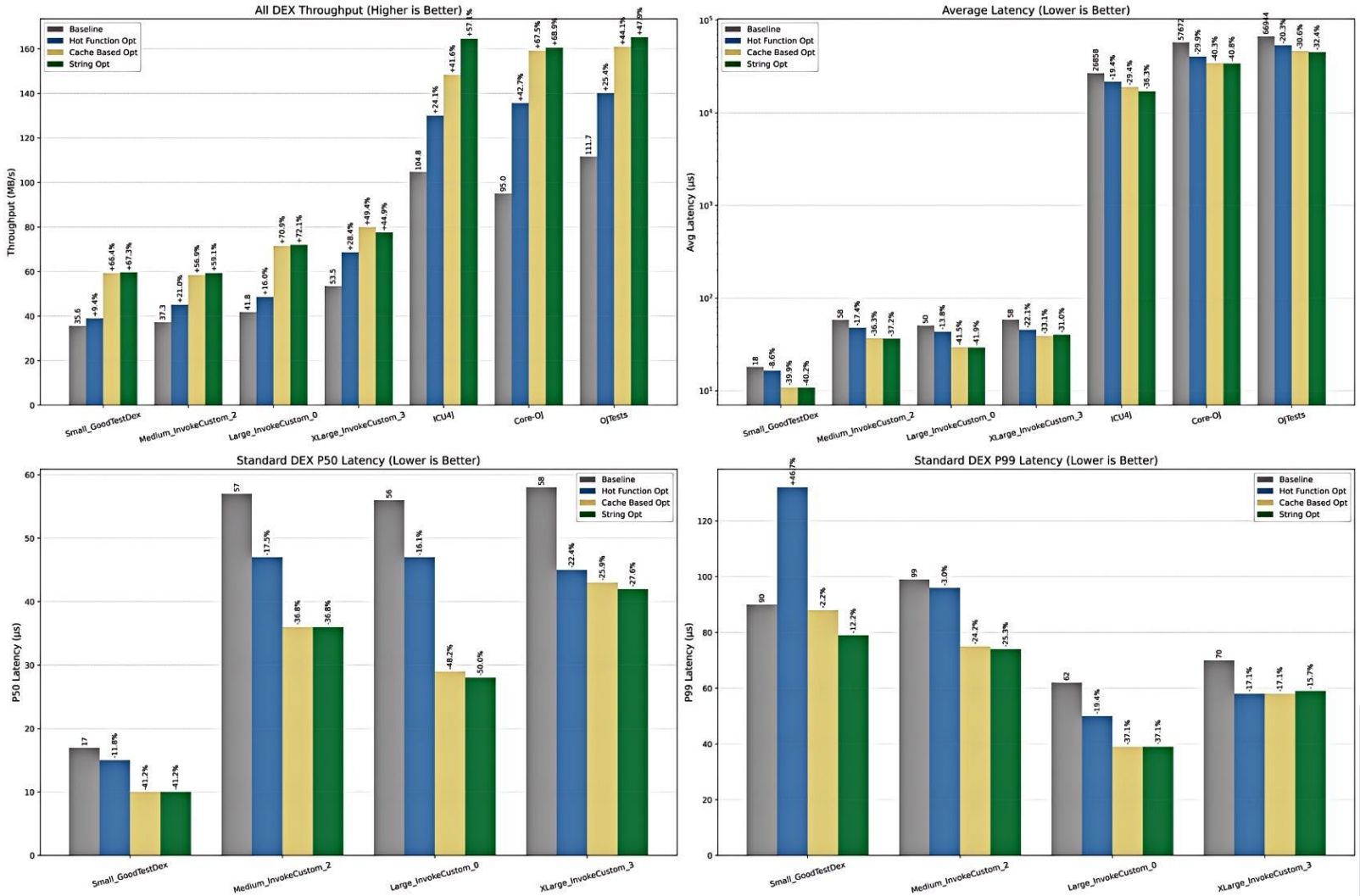
● UTF-8 验证查表法

- switch-case 判断 UTF-8 字节类型，分支多（5+ 个分支），CPU 分支预测失败率高
- 将字节类型映射为 4-bit 查找表，单次内存访问替代多重分支判断
- 消除分支预测失败、查表操作 L1 缓存命中

● ASCII 快速路径优化

- DEX 文件中 ~80% 的字符串是 ASCII，原代码对所有字符都执行完整 UTF-8 解码（GetUtf16FromUtf8），开销大
- 使用位运算 `(c1 | c2) < 0x80` 快速判断是否为 ASCII，ASCII 情况下直接字节比较，避免解码，仅在非 ASCII 时才执行完整解码

基于查找表的字符串处理



本次优化后

- 吞吐: ↑ 59.6%
- 延迟: ↓ 37.1%
- p50: ↓ 38.9%
- p99: ↓ 22.6%

上次优化后

- 吞吐量: ↑ 56.7%
- 延迟: ↓ 35.9%
- p50: ↓ 38%
- p99: ↓ 20%

LEB128 快速路径优化

LEB128 解码优化举例

核心思想： 赌大概率事件，减少安全检查

原版实现 (Slow Path)

```
while (true) {
  if (ptr >= end) // ⚠检查越界
    return error;
  byte = *ptr++;
  result |= ...;
  if (byte & 0x80 == 0)
    break; // ⚠跳转循环是否结束
}
```

每字节都检查边界+循环跳转

A. 批量边界检查

一次检查 5 字节
后续直接读内存

B. 循环展开

平铺 1~5 字节处理
消除跳转开销

C. 分支预测优化

LIKELY 宏提示
99% 数据 ≤2 字节

优化实现 (Fast Path)

```
if (ptr + 5 <= end) { // ✓一次检查
  b1 = *ptr++; if (done) return;
  b2 = *ptr++; if (done) return;
  b3 = *ptr++; if (done) return;
  ... // ✓线性展开
}
```

🚀 批量检查 + 无循环 + 分支预测

数据分布统计

- 1~2 字节: 99%
- 3~5 字节: 1%

→ *优先优化高频场景!*

CPU 流水线效果

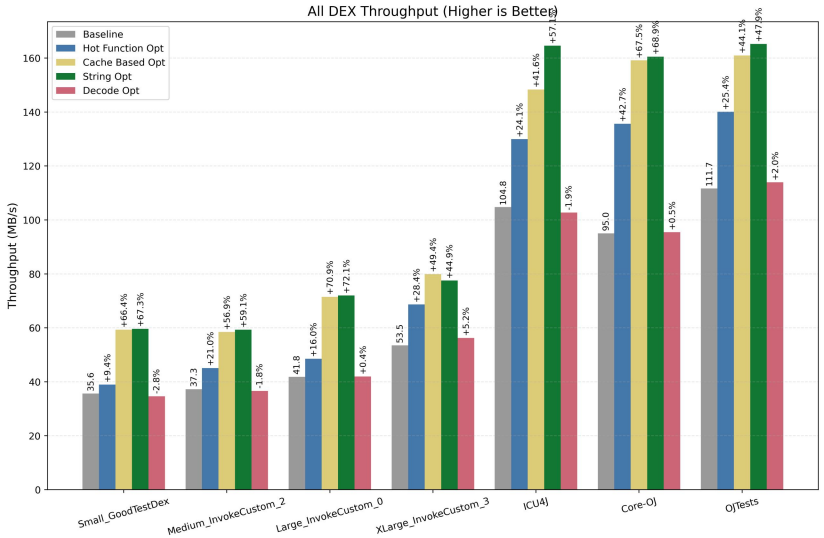
- 原版: 跳转 → 流水线打断 ❌
- 优化: 线性执行 → 预测准确 ✅

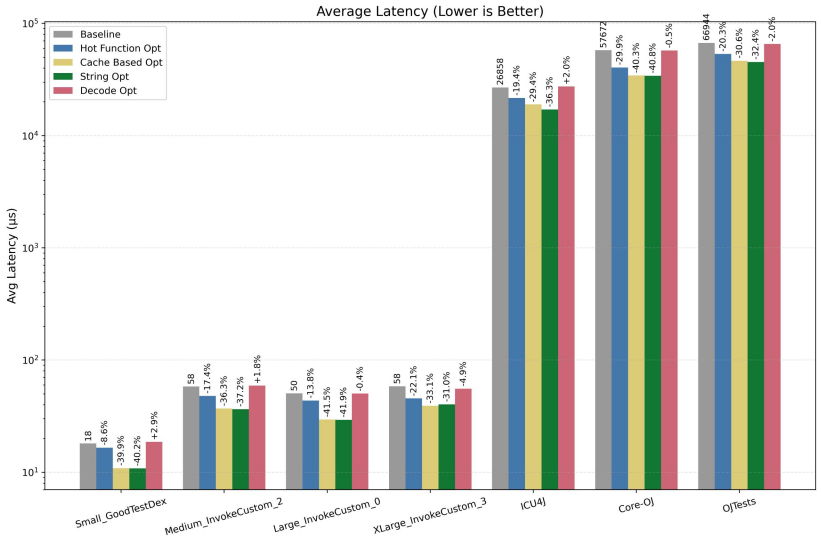
适用场景

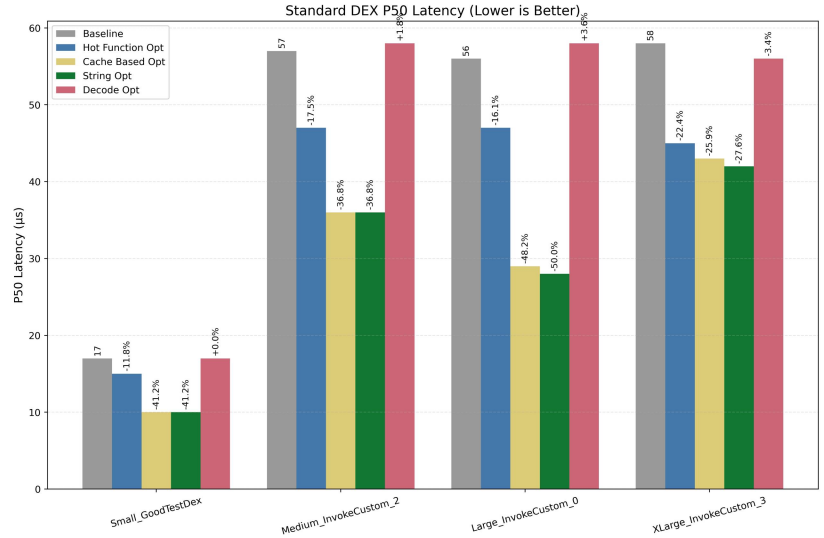
- Fast Path: 文件中间 99.9% 区域 (数据充足，数值小)
- Slow Path: 文件末尾最后几字节 (边界情况兜底)

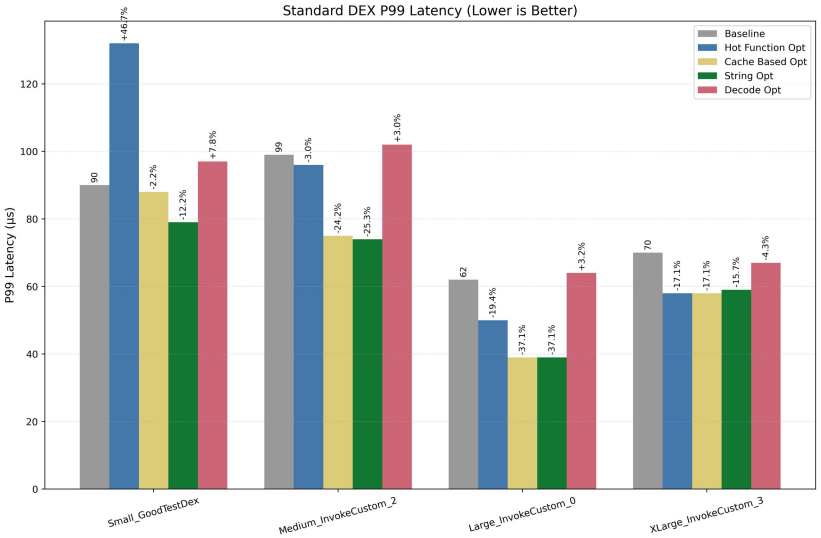
💡 **Fast Path 设计：为 99% 的常见情况提供快速路径，复杂情况由慢速路径兜底**

LEB128 快速路径优化效果









相对baseline

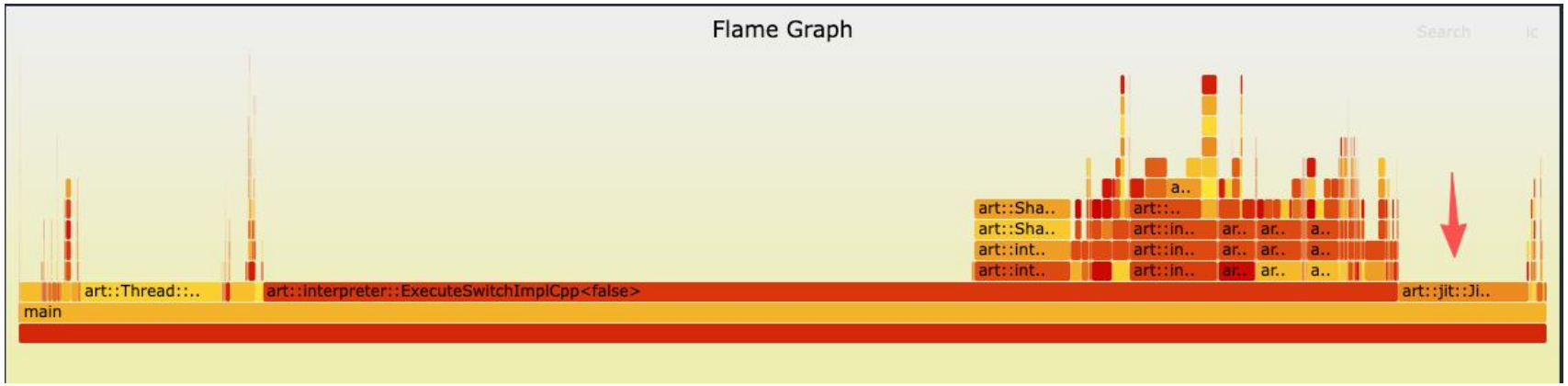
- 吞吐量: ↑ **0.2%**
- 平均延迟: ↓ **0.2%**
- **p50: ↑ 0.5%**
- **p99: ↑ 2.4%**

优化效果差

LEB128 解码使用很少

jit模块

- jit::Jit::MaybeDoOnStackReplacement（存在回边时检查jit代码是否存在）的执行占比很高，执行次数上kw次

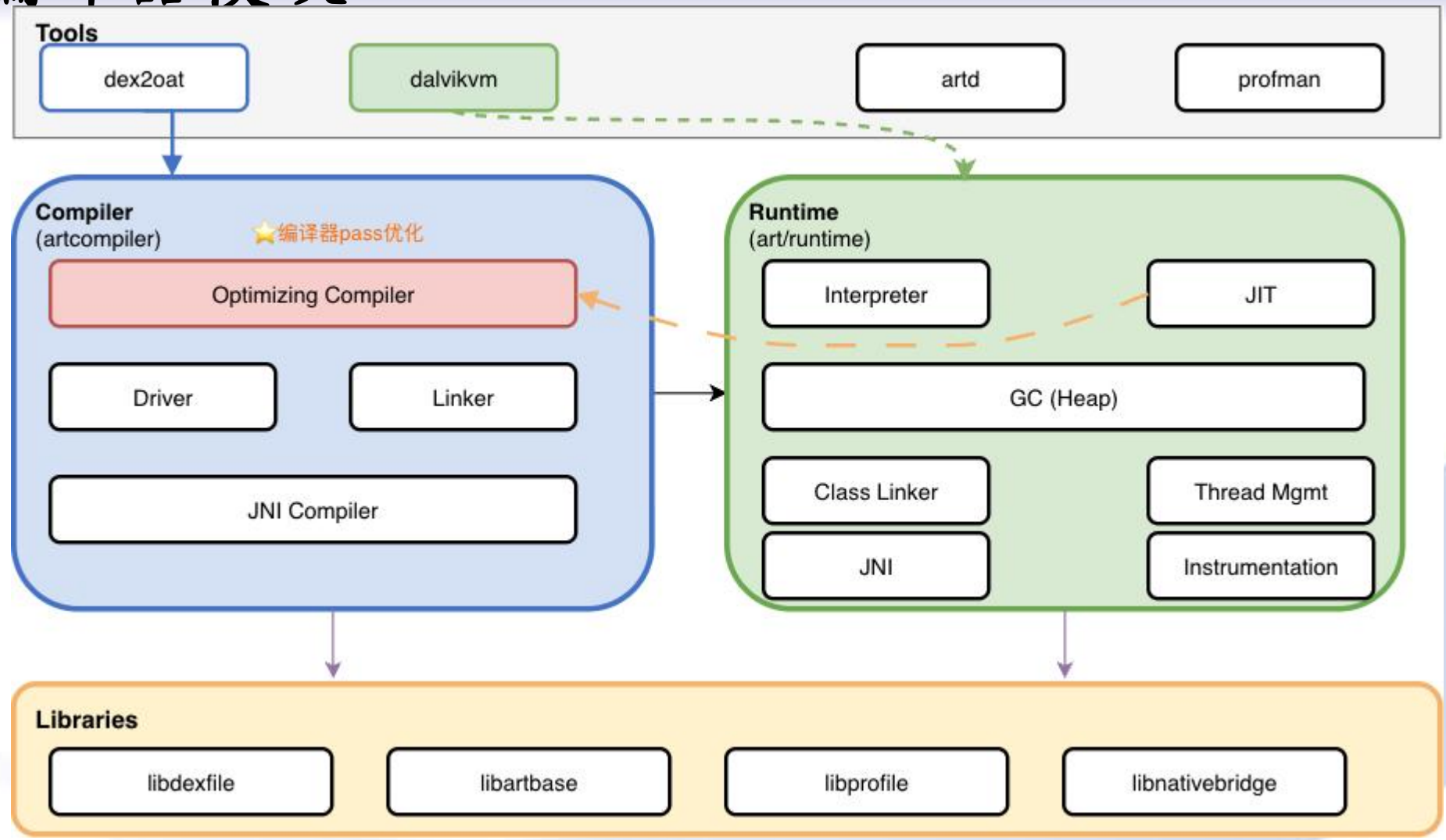


aosp15.la [SSH: la55] - jit-inl.h

```
inline bool Jit::MaybeDoOnStackReplacement(
    Thread* thread, ArtMethod* method, uint32_t dex_pc, int32_t dex_pc_offset, JValue* result) {
  Jit* jit = Runtime::Current()->GetJit();
  if (jit == nullptr) {
    return false;
  }
  return MaybeDoOnStackReplacementInternal(jit, thread, method, dex_pc, dex_pc_offset, result);
}
```

scimark跑分:49->56

编译器模块


Tools
dex2oat
dalvikvm
artd
profman
Compiler
(artcompiler)
编译器pass优化
Optimizing Compiler
Driver
Linker
JNI Compiler
Runtime
(art/runtime)
Interpreter
JIT
GC (Heap)
Class Linker
Thread Mgmt
JNI
Instrumentation
Libraries
libdexfile
libartbase
libprofile
libnativebridge

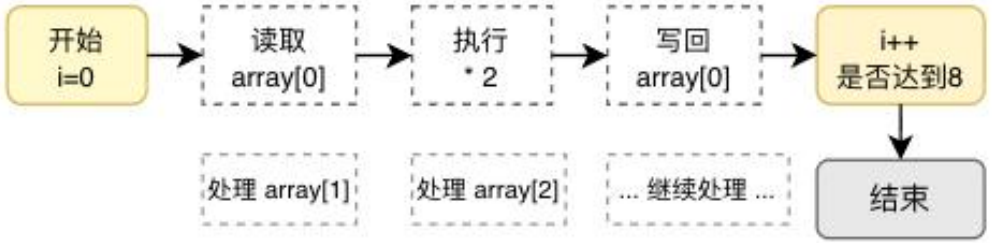
编译器Pass优化

循环向量化优化 (Loop Vectorization) - Android ART Runtime 性能优化技术

优化前：标量处理

```
for (i = 0; i < 8; i++) {
  array[i] = array[i] * 2;
}
```

这个元素串行处理
每次处理 1 个数据
执行 8 次循环



优化后：SIMD 向量化

```
for (i = 0; i < 8; i += 4) {
  vec_mul(array[i:i+4], 2);
}
```

4个元素并行处理
每次处理 4 个数据
只需 2 次循环

性能对比 & 关键技术

理论性能提升：2-16倍
向量宽度（4/816）
内存对齐

ARM NEON/ LA LSX
128/256-bit 向量寄存器
支持整数/浮点运算

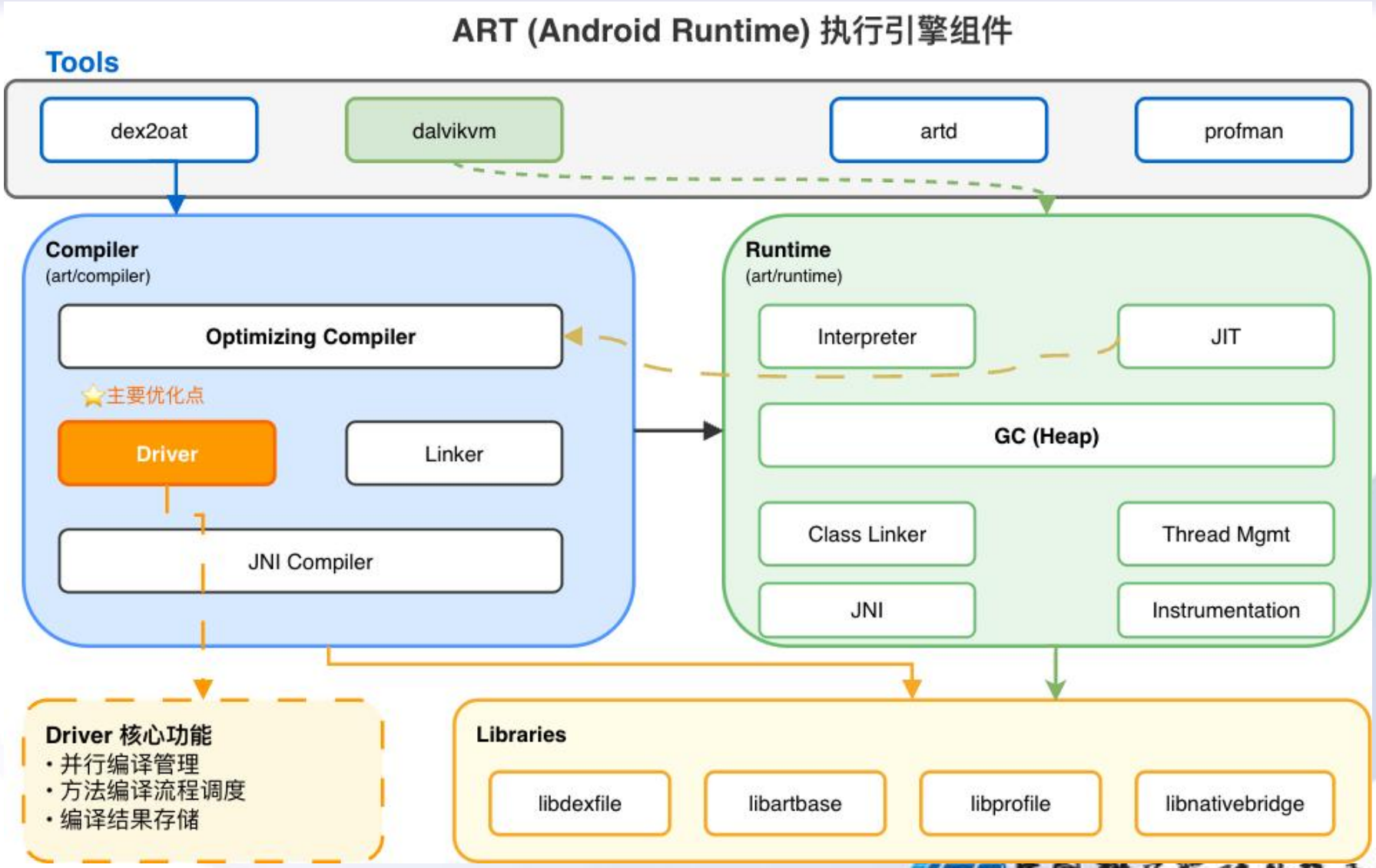
ART 编译器优化
自动检测可向量化循环
生成目标平台 SIMD 指令

典型应用场景
图像/视频处理、音频编解码
矩阵运算、快速傅里叶变换 (FFT)

提纲

三、下一步计划

下一步工作计划


ART (Android Runtime) 执行引擎组件
Tools
dex2oat
dalvikvm
artd
profman
Compiler
(art/compiler)
Optimizing Compiler
主要优化点
Driver
Linker
JNI Compiler
Runtime
(art/runtime)
Interpreter
JIT
GC (Heap)
Class Linker
Thread Mgmt
JNI
Instrumentation
Driver 核心功能
• 并行编译管理
• 方法编译流程调度
• 编译结果存储
Libraries
libdexfile
libartbase
libprofile
libnativebridge

感谢聆听！请批评指导！